人的实践经验形成过程的计算建模：证据、修订、复用与宿主智能体验证

Computational Modeling of Human Experience Formation through Practice: Evidence, Revision, Reuse, and Host-Agent Validation

许伟丨电子科学与技术专业，工学学士丨独立研究者，中国

摘要—人不是因为保存了一段文字就获得经验。经验来自目标驱动的行动、行动结果、结果解释、反例检验和后续复用；其中任何一步缺失，都可能把偶然成功误记成规律。本文以这一人类实践过程为研究对象，建立“实践—观察—解释—验证—经验—再实践”的计算模型。模型将经验表示为带主张、条件、版本、来源、证据、反例和生命周期状态的对象，区分候选、稳定、争议和失效；通过持久化版本链、来源约束、冲突记录、unknown 返回和宿主上下文恢复，使经验能够在下一次行动前接受条件检查。Beta-Bernoulli 更新用于描述有限证据下的可信度变化，信息增益用于选择值得求证的行动，Thompson Sampling 仅作为候选行动探索的可选控制，不把整个系统误写成贝叶斯强化学习。本文首先逐步说明人的经验形成逻辑，再给出状态机、更新算法、恢复算法和复杂度，最后报告 10 种子、12 轮、7 条件的机制实验、20 种子的端到端门槛实验以及 10 种子贝叶斯 Bandit 控制。规则复用基准中 B2 条件相对精确索引基线完成率提高 4.51 个百分点，源读取减少 81.60%，但本地逻辑操作增加 41.53%且时延没有优势。端到端门槛实验把经验形成与任务收益放进同一账本：把候选门槛放回循环后，相对精确索引仍多完成 4.20 个百分点并少读 73.33%来源，代价是每种种子多读 12 次；状态闸只在“尚未完成第二次独立实践”的坏主张上兑现价值（0 次错误对 209 次错误）。真实宿主验证由真实模型经 MCP 协议自主决定使用经验还是读取来源，七档预注册实验共 912 次作答：版本准入与坏经验识别成立；内容正确却被一次失败观察误判为争议时，答案不会变错，但复用收益整块归零且不会自愈（累计成功从 3 增至 6 仍停留在争议状态）；经内容级修订并附新证据可回到可复用；库内另存 300 条同族无关记忆时，召回仍每次只取回目标条目；在连续漂移下把召回的版本与状态元数据抹掉作消融对照，同一宿主、同一批记忆的正确率由 12/12 跌到 3/12，9 个错答逐个精确等于旧版本规则的答案，而无漂移的对照组两臂各 12 题全对；把同一消融交给两个外部 API 模型当宿主重跑 96 次作答，四格读数逐格复现，且在内容不含任何版本字符的干净子群上差异同样成立。最后直接把工作流换成“用与不用”的对照：一臂装配经验库（召回加按需读源），另一臂完全没有记忆层、只能读源；不限额度时两臂正确率相同，而前者读源次数由 72 次降到 20 次、提示词 token 降到 32%，把每臂读源额度收紧到 6 次与 12 次后，前者分别完成 18 题与 24 题、后者只完成 6 题与 12 题，两个跨族外部宿主逐格相同；记忆层的陈旧标记在三个漂移档上零误花零漏读，额度耗尽时返回 unknown 而不猜测。第七档把库清空、由宿主自己建档与改档：两臂各 72 题正确率相同，读源 36 次对 72 次（仅看库建成后的两轮为 12 次对 48 次），提示词 token 为对手的 59.2%；建档 24 次、改档 12 次与实际改版逐条对应，污染与陈旧复用各 0 次。结果支持限定契约任务中的经验复用机制，不证明模型参数自进化或普遍提升语言模型能力。

关键词—实践经验；经验生命周期；证据约束；版本修订；宿主智能体；贝叶斯证据更新

作者与致谢

作者许伟，电子科学与技术专业，工学学士。本研究于毕业后独立开展。感谢盐城工学院在学习期间提供的实验室环境、课程训练和知识培养；学校信息仅表示教育背景与致谢，不代表作者单位或论文研究机构。

I. 引言：经验不是文本，而是被实践检验的认识

人在面对问题时，通常先有目标，再根据已有认识采取行动。行动产生结果，结果与预期比较后形成反馈；反馈并不会自动成为经验，因为结果可能来自偶然、条件可能没有记录、观察可能被误读，或者相同成功在另一版本环境中已经失效。人类经验的关键不是“记住发生过什么”，而是把行动结果整理成在明确条件下可再次使用、可被反驳、可被修订的认识。现有智能体记忆系统往往强调存储和召回，却容易跳过经验是否成立、何时失效以及错误如何回收的问题[1-2]。

本文的主题是对人通过实践获取经验的过程进行计算建模。系统名为补天，依附宿主 AI 应用运行；宿主提供模型、会话和工具，补天负责经验对象、证据链、版本准入、状态转移、反馈写回和恢复上下文。本文不把软件记忆等同于人的意识，也不把文字记录等同于参数学习。人的实践逻辑被抽象成可观测工程过程，抽象是否有效由状态、证据、错误复发、复用收益和恢复质量来检验。

A. 研究问题与贡献

RQ1：人的“行动—结果—反思—验证—复用”过程能否表示为保持证据和条件的状态机？RQ2：候选经验何时可以稳定使用，何时必须返回 unknown 或进入争议？RQ3：在版本变化、资料中断和服务重启下，经验机制是否减少重复取证并避免旧规则错误？RQ4：贝叶斯更新与探索策略在这套经验模型中各自承担什么作用？本文贡献是：给出实践经验的分层定义；建立从证据产生到经验修订的完整算法；用真实 ButianService 完成多轮机制实验；用独立控制实验验证数值组件；明确宿主 AI 实测与机制实验的证据边界。

B. 相关工作与问题定位

经验学习研究把具体经历、反思、概念形成和后续试验联系起来。Morris 对 Kolb 模型的系统综述进一步强调经历的情境和反思的重要性[3]。本文据此把“做过”与“形成可复用经验”分开：单次经历是事件，解释是候选主张，后续实践才提供再次支持或反例。这一关系属于研究设计来源，不能直接推导软件中的两次成功阈值；阈值是可审计的工程选择，需要通过具体任务族进行敏感性评估。

智能体研究提供另一侧参照。Reflexion 把任务反馈整理成语言反思，Voyager 通过可复用技能支持后续探索[4-5]；MemoryBank 关注长期记忆，LongMemEval 评测跨会话记忆能力[1-2]。这些研究并不能替本文证明状态门槛和版本规则正确。本文将来源可追溯、证据不重复计数、版本变化后的重新求证以及反例后的争议保留放在同一实践过程

内，实验分别检查这些环节的作用。检索增强生成提供外部信息访问方式[6]，但召回相关文本与确认经验适用之间仍需要条件判断。

C. 论文推进顺序

全文按人的认识过程推进。第二节定义一次实践中发生了什么；第三节说明如何把实践结果变成候选经验；第三节说明证据、反例和版本如何决定经验能否复用；第四节给出算法与实现；第五节报告实验设计；第六节报告结果；第七节讨论贝叶斯和探索机制的位置、工程收益与限制；第八节给出结论。数学工具服务于经验形成主线，而不是替代主线。

II. 人通过实践形成经验的过程模型

A. 从目标到行动结果

设人在时刻t面对任务目标G_t、环境E_t和已有认识M_t。人先提出行动方案A_t，再通过工具或身体行动改变环境，获得观察O_t。预期结果$\hat{O}_t$与实际结果O_t之间的差异产生反馈$F_t = O_t - \hat{O}_t$。这个差异不是简单的“对/错”标签：如果行动没有真正执行，结果不可解释；如果环境版本未知，成功不能迁移；如果观测没有来源，结果不能复核；如果结果只发生一次，统计稳定性仍然不足。计算模型因此保留行动、环境快照、观察来源和结果证据，而不是只保存一句总结。

$$(G_t, E_t, M_t, A_t) \rightarrow O_t \rightarrow F_t \tag{1}$$

B. 从反馈到候选认识

人在反思阶段会尝试回答三个问题：发生了什么，为什么发生，下一次在什么条件下可以复用。对应的候选经验 e 包括主张 h、适用条件 c、环境版本 v、来源 s、证据集合 E 和反例集合 X。候选经验不是事实，它只是对实践结果的可检验解释。若只有结果没有条件，候选过于宽泛；若只有主观解释没有来源，候选不能审计；若反例被删除，候选不能说明适用范围。

$$e = (h, c, v, s, E, X, l, \theta) \tag{2}$$

其中 l 是生命周期状态，θ 是证据支持下的可信度摘要。θ 不是真理概率；它是当前证据和工作先验下的估计。一个经验只有在后续实践中再次预测成功、并且没有未处理的反例时，才可能从 candidate 进入 warm。

C. 再实践与经验修订

下一次任务不是单纯读取 e，而是先检查当前任务条件是否满足 c 和 v，再决定是否采用。采用后必须冻结首次行动结果，防止事后答案污染首次评价。若结果符合预测，追加带独立键和证据引用的成功观察；若失败，追加反例并进入 disputed 或触发修订；若资料、工具或停止状态无法确认，返回 unknown。修订不是覆盖旧文本，而是在版本链上产生新版本并保留父版本、变更原因和新证据。

D. 一个贯穿示例：校准规则如何形成经验

考虑某传感器在版本 v1 下采用线性换算。第一次任务给出实体编号、软件版本和原始读数，宿主依据资料选用规则并产生输出；仪器校准记录是结果证据。系统不把“这次答对了”直接写成普遍规律，而是形成“在实体 a、版本 v1 及给定量程下，规则 h 有效”的 candidate。第二次使用不同校准样本、不同证据摘要再次成功，才满足局部门槛。即使两条文件不同，也可能来自同一错误来源；去重只能排除已识别重复，不能保证统计独立。

当软件变为 v2，旧经验的条件不再覆盖当前任务。系统应先找 v2 来源；若来源中断，则保留 unknown，而不是用 v1 规则猜测。来源恢复后生成新版本，并通过后续实践检验。若 v1 经验在匹配条件下失败，反例必须附着在原主张上，状态进入 disputed；另写一段更自信的解释不能自动消除失败。这样同一个例子串起目标、行动、证据、候选、复用、漂移和修订，也说明论文所研究的对象是一条过程。

E. 五个不可跳过的过程环节

第一是目标化：没有要解决的问题，就没有经验的适用任务。第二是行动化：经验必须能够约束实际行动，而不是只是一段解释。第三是结果化：必须观察行动后果并保留证据。第四是验证化：要检查条件、反例、版本和重复观察。第五是再实践化：经验必须在后续任务中被调用并接受结果。补天的软件状态机、证据表、版本表、上下文恢复和反馈接口分别对应这五个环节。缺任何一个环节，都只能称为日志、检索结果或候选主张。

III. 经验生命周期：从候选到可复用认识

A. 状态定义

candidate 表示由一次或少量实践产生、尚未达到稳定使用条件的候选经验；warm 表示达到既定成功和证据门槛、可以在匹配条件下参与召回的经验；hot 表示近期持续有效且被频繁使用的经验，但热度不是正确性的额外证明；disputed 表示出现反例、冲突或环境不匹配，默认不能无条件使用；invalidated 表示已知不再成立；unknown 表示一次观察或任务的结果尚不能确定；它不是与 candidate 等并列的经验热度状态。实际记忆还包含 cold、archived 和 pending_validation 等维护状态，本文选取与证据形成直接相关的状态解释。状态名描述程序记录，不描述模型的心理状态。

$$s \geq 2, f = 0 : \text{candidate} \rightarrow \text{warm} \tag{3}$$

$$\text{failure} : \text{warm} \rightarrow \text{disputed} \tag{4}$$

B. 经验进入系统的条件

外部文章或宿主回答只能产生候选来源，不能直接成为本地稳定经验。进入系统至少需要主张、适用任务、来源事件、环境或版本信息和可追溯的结果。success 必须携带 evidence_ref；independent_key、证据摘要和可选正文哈希用于去重。重复同一文件不能伪装成多次独立实践。

C. 经验复用的准入顺序

准入顺序是：先识别任务实体和目标版本，再召回候选；检查候选是否唯一、正文是否完整、适用条件是否满足、版本父链是否连续；若冲突或预算截断，回退精确来源；若来源不可用，返回 unknown。这个顺序模拟人在使用经验前回忆情境、比较当前条件和检查可靠性的过程。生产 compose_context 只提供候选，研究解析器和宿主负责最终准入，不能把召回本身写成理解。

D. 失败如何改变经验

失败不是把经验静默删除。系统保存失败证据、失败环境、当前版本和首次答案；成熟经验的失败进入 disputed，候选经验的失败保留在观察记录中并阻止无条件升级。若发现主张需要缩小条件，生成新版本；若发现主张完全不成立，进入 invalidated。这样保留“过去为什么相信”和“现在为什么修订”，对应人通过反例修正认识的过程。

IV. 算法与实现：把实践过程变成可执行流程

A. 算法 1：实践记录与候选生成

输入任务G、行动A、环境快照E、观察O、来源s和预期$\hat{O}$。首先计算结果状态；若行动没有完成或证据缺失，记录 unknown 并停止升级。若结果成功，创建或读取经验对象，检查独立键、证据摘要和正文哈希；重复证据拒绝。若结果失败，写入反例并把状态置为 disputed 或保留 candidate。MemoryStore 中的观察、证据摘要、经验版本和领域技能更新在同一个 BEGIN IMMEDIATE 事务中提交；ButianService 随后向另一服务存储追加 practice_observed 事件。两步没有跨库原子提交，不能宣称服务事件与记忆事务具有整体原子性。

$$r_t \in \{\text{success}, \text{failure}, \text{unknown}\} \tag{5}$$

B. 算法 2：经验可信度的 Beta 更新

在经验主张“下一次满足条件时成功”的工作模型下，令$\theta \sim \text{Beta}(\alpha, \beta)$。一次成功证据权重为$w = qdm$，其中 q 是证据质量、d 是独立性评价、m 是环境匹配度；失败证据采用同样权重。成功更新$\alpha' = \alpha + w$、$\beta' = \beta$，失败更新$\alpha' = \alpha$、$\beta' = \beta + w$。后验均值$\mu = \alpha/(\alpha+\beta)$，成功带来的变化为$\Delta\mu = \frac{w\beta}{(\alpha+\beta)(\alpha+\beta+w)}$。因此成功会提高当前工作信念，且在参数增大后边际变化下降；这描述证据积累，不证明主张真实。

$$\theta \sim \text{Beta}(\alpha, \beta), \mu = \frac{\alpha}{\alpha+\beta} \tag{6}$$

当前 record_practice_outcome 的生产 confidence 采用有界增减规则：失败减 0.1，候选达到升级门槛时加 0.1；该字段没有自动调用 BetaBelief。因此本节 Beta 模型是数学模块与控制实验支持的可选表征，不能把生产 confidence 当成 Beta 后验。分数权重是加权似然的工程近似，不能自动解决重复证据；d 必须由上层去重和环境审查支持。Beta 后验不能越过版本准入和实践门槛，不能把 warm 状态直接解释为“已证明”。

C. 算法 3：候选经验选择与信息增益

当多个候选行动都可能减少不确定性时，信息增益$\text{IG} = H(p_{\text{prior}}) - H(p_{\text{posterior}})$可用于排序，成本惩罚后的分数为$\text{IG} - \lambda\text{cost}$。只有后验集合满足概率归一和全概率混合，期望信息增益才具有互信息解释。补天的 expected_information_gain 检查显式后验维度、结果概率和混合一致性；不从文字自动推导后验。

$$\text{EIG} = H(p) - \sum_y r_y H(p(\cdot|y)) \tag{7}$$

D. 算法 4：Thompson Sampling 的正确位置

Thompson Sampling 不是经验形成过程本身，而是当系统有多个待验证行动或候选策略时的探索分配器。每个候选臂k维护$\text{Beta}(\alpha_k, \beta_k)$，每轮采样$\tilde{\theta}_k \sim \text{Beta}(\alpha_k, \beta_k)$，选择最大者，观察 Bernoulli 反馈后只更新被选臂。其选择复杂度为 O(K)，更新为 O(1)。当前代码没有状态转移、长期回报或价值函数，因此它是上下文无关的 Bayesian bandit，不是完整 Bayesian reinforcement learning。

E. 算法 5：服务重启后的经验恢复

恢复时先读取已提交事件、经验当前版本、父版本链和未完成任务，再恢复候选上下文。若事件处于 prepared 但没有 committed，不能把外部行动当成成功；若工具退出没有确认，状态保留 unknown；若当前版本和经验版本不一致，经验只能作为候选并触发重新取证。恢复算法的目标是延续实践过程，而不是简单把旧文本重新塞回上下文。

F. 带输入输出与错误分支的伪代码

以下伪代码按实际责任边界编写。算法 1a 对应生产实践接口；算法 2a、3a、4a 对应数学模块；算法 5a 是宿主侧恢复与准入流程，不能当作一个已实现的单函数；算法 6 是主实验真实执行路径。算法 6 直接写 warm，不能代替算法 1a 的候选验证。行号表示执行顺序，IF/ELSE 和 FOR 块用 END 闭合。

算法 1a 记录实践反馈（生产接口）

```
01 INPUT: member, memory_id, content, outcome, independent_key, evidence_ref
02 OUTPUT: memory_id, status, successes, failures；或明确错误
03 REQUIRE outcome IN {success, failure, unknown} 且 independent_key 非空
04 IF success 且无 evidence_ref: RAISE practice_success_evidence_required
05 由服务验证证据归属与正文哈希；无效则 RAISE 并结束
06 BEGIN IMMEDIATE（成员记忆数据库事务）
07 IF 独立键已用，或证据引用/摘要已用: RAISE 重复证据错误
08 读取经验；不存在则创建 candidate；如指定正文哈希则校验
09 插入观察、环境和证据摘要；统计该 memory_id 的成功数 s 与失败数 f
10 IF outcome=failure: 成熟经验置 disputed，否则保留 candidate；降低 confidence 与 weight
11 ELSE IF outcome=success AND s>=2 AND f=0 AND 旧状态为 candidate 或 pending_validation: 置 warm 并有界增加分值
12 ELSE: 不执行升级；unknown 不计入成功或失败
13 保存新版本、父链、来源引用和领域技能统计；COMMIT
14 ON 事务内异常: ROLLBACK；重新抛出错误
15 事务后服务另行追加 practice_observed 事件；失败不得宣称两库整体回滚
16 RETURN 实际已提交状态与计数
```

算法 2a Beta 证据摘要（可选数学模块）

```
01 INPUT: alpha>0, beta>0, outcome, q,d,m IN [0,1]
02 OUTPUT: 更新参数与后验均值；不改变生产生命周期状态
03 检查参数有限性与范围；不合法则 RAISE
04 w <- q*d*m
05 IF outcome=success: alpha <- alpha+w
06 ELSE IF outcome=failure: beta <- beta+w
07 ELSE IF outcome=unknown: 保持参数
08 ELSE: RAISE invalid_outcome
09 RETURN alpha, beta, alpha/(alpha+beta)
10 NOTE: 去重在算法 1a 执行；分数权重不是独立性的证明
```

算法 3a 求证行动排序（显式后验控制）

01 INPUT: 先验 p；行动 a 的结果概率 r；各结果后验 p_y；成本 cost 与惩罚 lambda
02 OUTPUT: 合法行动的 EIG 与成本调整得分
03 FOR EACH action a:
04 检查 p、r、p_y 有限非负、维度一致、各分布归一
05 检查对每个分量 i 有 sum_y r_y*p_y[i] 与 p[i]在容差内一致
06 不合法则 RAISE，不凭文字猜测后验
07 eig[a] <- H(p)-sum_y r_y*H(p_y)
08 score[a] <- eig[a]-lambda*cost[a]
09 END FOR
10 RETURN 按 score 排序的行动；执行选择仍由调用方决定

算法 4a Thompson 探索分配（Bandit 控制）

01 INPUT: K 个臂的正参数 alpha[k], beta[k]与随机数生成器
02 OUTPUT: 选择臂 k；获得反馈后更新该臂
03 REQUIRE K>0；所有参数有限且为正
04 FOR k=1 TO K: sample[k] <- BetaSample(alpha[k],beta[k])
05 END FOR
06 selected <- ARGMAX sample[k]（使用实现约定处理并列）
07 执行 selected；等待可核验的 Bernoulli 反馈 reward
08 IF 反馈未知: 不伪造 reward，不执行成功或失败更新
09 ELSE IF reward=1: alpha[selected] <- alpha[selected]+1
10 ELSE IF reward=0: beta[selected] <- beta[selected]+1
11 ELSE: RAISE invalid_reward
12 RETURN selected 与更新参数；不据此自动晋升经验状态

算法 5a 恢复后的条件准入（宿主责任流程）

01 INPUT: 已持久化任务、事件、经验、版本链与当前任务条件
02 OUTPUT: 可复核候选上下文与结果状态；并非自动成功
03 打开服务存储与成员记忆，读取当前版本和来源
04 检查父版本链；损坏则报错，不补造缺失版本
05 FOR EACH 未完成行动:
06 committed 有可核验证据时才恢复其已提交结果
07 prepared 无提交、退出未确认或证据缺失时保留 unknown
08 END FOR
09 召回候选；再按实体、当前版本、完整性与冲突逐项准入
10 候选不适用则重新取源；来源不可用则 RETURN unknown
11 RETURN 候选、来源和未决事项；宿主仍需执行下一次实践
12 NOTE: 当前实验只验证正常 close/open，不验证任意断电恢复

算法 6 多轮规则复用基准（研究脚本实际路径）

01 INPUT: 共享任务流、精确来源档案、条件 method 与记忆目录
02 OUTPUT: 首次答案、正确/错误/unknown 及读取/查询/写入计数
03 IF 条件启用记忆: compose_context；解析唯一实体规则
04 IF 启用版本准入: 拒绝不匹配版本；否则允许旧规则
05 IF 无可用规则 AND source_available: 扫描或精确索引读取来源
06 answer <- 固定 solve(rule,task)；冻结首次答案
07 依据 reference 计分；此后修复不能改写首次分数
08 IF 首次错误 AND source_available: 重新取正确来源规则
09 IF 已取源 AND 条件允许写入: append_event；MemoryStore.put(status=warm,confidence=1)
10 按协议第 8 轮 close/open；no-persist 条件改用空目录
11 RETURN 本轮记录；后续轮重复上述步骤
12 NOTE: 本算法没有 candidate 验证、自动归纳或 disputed 状态实验

G. 做梦整理、假设验证与后续实践

做梦是对已有实践材料进行离线整理的过程。DreamRunner.run 在材料数量、新材料、间隔和预算满足门槛后生成来源快照；提炼、抽象与假设正文由宿主提供，系统不从文本自动推导规律。distill 保留原材料引用，abstract 保留支持、反例及适用条件，propose 检查来源是否仍有效。验证动作经过 schedule 和 start 后才能接收 supported 或 rejected 结果，证据需通过服务核验。supported 只产生 candidate；后续实践接口仍需累计成功才能进入 warm。验证并不保证任意任务上的正确性，其作用是把可检验假设与未经检验的概括分开。

算法 7 做梦形成与验证后复用

01 INPUT: 实践材料、宿主提出的候选规律、验证预算与后续任务
02 OUTPUT: supported/rejected/unknown、候选经验与后续实践记录
03 检查新材料、间隔与预算；不满足则跳过本次整理
04 固定来源快照；由宿主提炼摘要、抽象条件并提出假设
05 安排验证；授权且来源仍有效时启动动作
06 执行与训练样本不同的验证输入；保存观察及证据
07 IF 存在反例: 记录 rejected；后续任务回退来源
08 ELSE IF 证据不足: 记录 unknown；不当作已验证规律
09 ELSE: 记录 supported 并产生 candidate
10 后续任务采用候选时冻结首次结果并写回独立实践反馈
11 达到生产门槛后进入 warm；记录错误时保留反例
12 RETURN 验证与复用账本；不得将 supported 解释为普遍有效

H. 有限闭环的条件命题

命题 1：若每次状态更新都经 MemoryStore 实践接口，candidate 升级前必须存在至少两条通过接口去重的成功观察且无失败。证明直接来自升级分支：程序在事务内读取成功数和失败数，仅在 successes≥2、failures=0 且旧状态属于 candidate 或 pending_validation 时置 warm。该命题证明的是程序门槛，无法证明外部证据真实，也无法保证绕过接口的人工写入满足同样条件。两次成功是必要的接口条件，不能作为经验普遍有效的充分条件。

命题 2：若当前任务版本可确定，且宿主严格执行版本准入，旧版本候选不能直接产生当前版本的已接受结果。证明分两种情况：候选版本匹配时进入正常求解；不匹配时必须重新取证或返回 unknown。该条件命题的关键假设是版本识别和准入确实执行；no-version 消融移除该假设，因此产生的错误用于说明守卫的作用，不是现实场景永不出错的证明。

命题 3：一次 unknown 观察不应计入成功数或失败数。代码只对 outcome 等于 success 或 failure 的记录进行相应计数，unknown 可以保留观察和版本记录，但不能据此增加支持样本。由此可避免资料断开或停止未确认时把缺失结果解释为成功。未知次数上升仍会降低任务完成率，因此 unknown 是诚实记录不确定性的代价，不是可以从评测分母中移走的无效样本。

过程闭环成立还需要宿主实际执行行动、返回可靠观察并在下一轮使用记录。数据库写入成功只能证明保存完成；主张在多个任务上有效需要外部结果，长期收益需要跨轮对照。本文将可执行闭环与认识正确性分开论证：前者由接口、事务和恢复控制验证，后者只在给定实验任务族内报告结果。

I. 复杂度与实现边界

经验写入涉及固定种类的表操作，但成功计数、失败计数和领域汇总会随观察记录增长；令N为相关观察数、N_d为领域观察数，聚合在无更强索引保证时保守计为$O(N+N_d)$，不能把整条实践写回称为 O(1)。固定表操作数量也不意味着每条 SQL 耗时为常数；候选召回使用当前实现的 OR LIKE 路径，若 M 条经验、平均正文长度 L，保守成本为 O(ML)；候选解析为 O(B)，B 为上下文字符数；版

本链检查为 O(V)，V 为版本数；Beta 更新为 O(1)；Thompson 一次选择为 O(K)；Pareto 预算分配取决于前沿状态数并设 100000 上限。复杂度说明的是程序执行量，不等于宿主模型 token 成本，也不等于人的认知复杂度。

V. 实验设计：检验经验过程而非只检验公式

A. 实验对象与问题

主实验是规则复用基准，并未验证完整经验形成过程。脚本 remember 直接调用 MemoryStore.put 把精确来源规则写为 warm、confidence=1.0，没有执行 candidate 门槛、独立实践检验或 disputed 反例处理。任务生成器给出实体和版本，固定求解器按规则计算结果，评测器比较参考答案，随后缓存来源规则以供下轮复用。实验不调用语言模型，因此测量的是经验基础设施是否能支撑实践循环，而不是语言生成能力。10 个种子、12 个实体、12 轮、7 个条件共 10080 次执行；第 5 和第 9 轮发生版本漂移，第 6 和第 7 轮发生资料中断，第 8 轮关闭并重启服务。

B. 条件与对照

B0-scan 每轮扫描档案；B0-index 使用精确版本索引；B1-static 首轮写入后不更新；B2-full 执行召回、版本检查、源回退、反馈写回和持久化；no-version 取消版本准入；no-write 不写经验；no-persist 在重启后使用空目录。首次结果在反馈前冻结，correct、wrong、unknown 互斥。统计单元是 10 个种子，不把重复轮次当独立样本。

C. 指标与假设

主要指标是首次完成率、版本错误数、unknown 数和源读取次数。次要指标是上下文查询、写回次数、逻辑操作、本地耗时和版本链审计。H1：在漂移和断源任务中，带版本准入的规则复用策略减少重复源读取并避免旧版本错误；H2：持久化在重启后保留可复用经验；H3：经验机制可能增加本地逻辑开销，因此源读取收益不等于端到端时延收益。

D. 生命周期接口控制及其局限

另一项 24 目录控制调用 record_practice_outcome：首次成功得到 candidate；同一证据换独立键被拒绝；第二份不同证据使状态变 warm；随后失败进入 disputed；脚本人工改写为 candidate；正常 close/open 后检查修订内容、父版本链和召回。它证明接口按设计执行状态守卫，不证明宿主会自主归纳、证据确实独立或新主张正确。证据文件由脚本构造，人工修订也由脚本明确指定，不能把 24 次相同流程称为 24 项科学发现。该控制与规则复用基准互补，原本尚未连接为一项同时测量经验形成和任务收益的端到端实验；这一缺口由§V.G 补齐。

E. 做梦效果的配对回放设计

新增实验以 10 个种子各生成线性、饱和两类传感器，共 20 组材料。每组初始观察为 x=1、2、3；当前宿主依据相邻差相同提出仿射候选，固定该归纳模板后由 Python 通过生产 dream 接口回放。关闭做梦条件逐题取源；未经验证条件直接采用候选；验证条件执行 x=5、10 两个新探针，有反例则回退来源，无反例则保存 candidate 并在后续实践中写回。后续测试为 x=4、7、9、12，重复 4 轮，每条件 320 题。验证输入与测试输入分离；三条件拥有相同初始材料，训练取源各 60 次。饱和族在 x 大于 6 后进入平台，初始三点无法区分两类。该设计用于观察有限材料的过度泛化以及求证成本，不是盲测模型自主发现实验，也不提供独立模型样本。

F. 独立算法控制

另行运行 10 个固定种子、每种子 1000 轮的两臂 Bernoulli 控制，真实奖励概率为 0.30 和 0.70，对比 Thompson、ε-greedy 和均匀随机。该实验只验证候选行动探索机制，actual_model_runs=0；不把它并入人的实践主实验，也不把它称为完整 Bayesian RL。

G. 端到端门槛实验设计

为把经验形成与任务收益连成一项实验，新增端到端门槛基准（脚本 run_endtoend_gate_study.py，判据在运行前写成协议并冻结）。它逐语句复刻主实验的任务流，因此任务、漂移、断源与重启完全相同；唯一变化是经验如何被承认为可复用。六条条件：G0 精确索引；G1 源规则首次取到即置 warm，即表 1 中 B2-full 的写入路径；G2 每次源读取记一次独立实践，需两次互不相同的证据才升温；G3 在 G2 上加一条状态为 candidate 的坏主张；G4 同 G3 但候选态也被视为可复用；G5 在 G2 上加一条带两次独立实践的坏主张。于是 G3 减 G4 分离状态闸的作用，G3 减 G5 分离第二次独立实践的作用，G2 减 G1 度量门槛的总代价，G2 减 G0 度量经验相对无经验的收益。

坏主张在第 6 和第 10 轮开始前注入到上一轮刚漂移的实体：内容取漂移前的旧规则，版本号改成当前版本，即版本自洽而内容过时。这是生产中最常见的坏主张形态，也是本实验唯一施加的外部冲击。20 个种子（原 10 个加新 10 个）、12 轮、12 实体，共 17280 次执行。复现自检要求 G1 在 10 个原种子上与已发布 B2-full 逐行一致，同机重复运行为逐字段一致；统计单元是种子，区间为种子簇 bootstrap。

VI. 结果：经验复用的收益与代价

A. 主实验结果

表 1 实践循环的首次结果与取证成本

条件	首次完成	错误	未知	源读取
B0-scan	1375/1440	0	65	42287
B0-index	1375/1440	0	65	1375
B1-static	1400/1440	0	40	752
B2-full	1440/1440	0	0	253
no-version	1307/1440	133	0	—
no-write	1375/1440	0	65	1375
no-persist	1440/1440	0	0	373

B2-full 是实验条件名称，表示规则复用策略的全部开关，并不表示完整经验生命周期。相对精确索引基线，其首次完成率提高 4.51 个百分点，种子聚类 bootstrap 95%区间为[3.68,5.35]。源读取从 1375 降至 253，减少 81.60%；无版本检查产生 133 次过期规则错误，说明版本不是附属字段，而是经验能否迁移的条件。重启丢失经验的条件增加 120 次源读取，说明持久化改变了后续实践所需的取证动作。

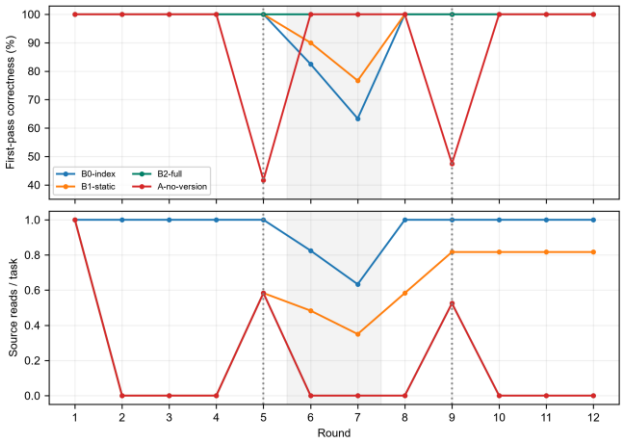


图 1 各轮漂移、断源和重启下的实践结果。

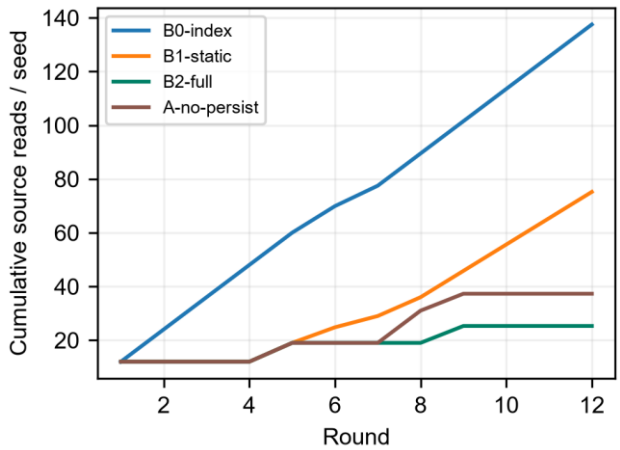


图 2 经验复用对累计来源访问的影响。

B. 逐轮结果：收益究竟从哪里产生

表 2a 按轮次分解首次结果和 B2 外部来源读取

轮次	索引正确/120	B2 正确/120	B2 源读取
1	120	120	120
2—4（每轮）	120	120	0
5（漂移）	120	120	70
6（断源）	99	120	0
7（断源）	76	120	0
8（重启）	120	120	0
9（漂移）	120	120	63
10—12（每轮）	120	120	0

第一轮两组都需要读取 120 份来源，因此没有冷启动优势。第二至第四轮规则不变，B2 复用已存规则，索引每轮仍需 120 次来源访问；两者正确率完全相同。第五轮 70 个实体更新，版本准入使 B2 拒绝 70 条旧规则并重新读源，剩余 50 条继续复用。第九轮同样重新读取 63 份来源。因此 B2 总源读取严格分解为 120+70+63=253；减少的 1122 次源读取来自后续重复任务。

完成率差异全部发生在第六、七轮：索引因来源不可用分别返回 21 次和 44 次 unknown，B2 利用此前已保存且版本匹配的规则完成这 65 题。来源可用的 1375 题两组均 1375/1375 正确；来源不可用的 65 题，索引 0/65 完成，B2 为 65/65。故 4.51 个百分点改善是这份协议下的信息可用性改善，并非推理能力、求解公式或语言理解能力变强。协议先让第五轮更新规则再在第六、七轮断源，给 B2 留下学习新版本的机会；未覆盖版本更新与首次断源同时发生的更困难情形。

取消版本准入后，第五轮 70 题、第九轮 63 题直接使用旧规则，共 133 次首次错误。反馈发生在首次答案冻结后；来源可用时脚本才用新规则纠正缓存，因此错误没有

延续至后续稳定轮。第八轮正常重启后 B2 仍有 120 次命中，而空目录策略需要重新读源 120 次；两者都 120/120 正确。持久化在本协议中的收益是避免重读，不是提高该轮正确率，也不是断电恢复保证。

C. 操作代价与边界

表 2 经验形成与复用的操作账本

条件	源读取 R	召回 Q	写回 W	逻辑操作
精确索引	1375	0	0	1375
B2-full	253	1440	253	1946
B1-static	752	1440	120	2312
no-persist	373	1440	373	2186

B2-full 逻辑操作比索引基线增加 41.53%，平均本地耗时 8.44 ms，而索引约 0.0102 ms。经验机制减少的是源取证和旧规则错误，不保证本地时延下降；该时延只测本地基准代码，不能据此解释真实人的学习成本。按等权逻辑操作计数，规则复用没有总操作优势；只有外部来源读取比本地召回和写回昂贵时，减少来源访问才可能抵消额外管理成本。当前没有测量外部服务费用、网络延迟或模型 token，因而不能给出真实应用的盈亏阈值。

D. 独立 Bayesian Bandit 控制

表 3 候选行动探索控制

策略	平均总奖励	平均伪遗憾	种子数
Thompson	704.4	4.32	10
ε-greedy	672.8	25.44	10
Uniform	494.3	202.60	10

该实验表明在两臂平稳 Bernoulli 环境中，Thompson 控制能较快集中到高奖励臂。它只说明探索分配器的行为，不说明经验主张已经正确，也不说明宿主模型获得了强化学习能力。

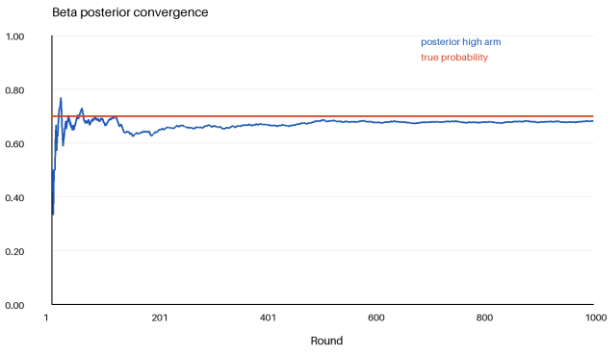


图 3 高奖励候选的 Beta 后验均值收敛。

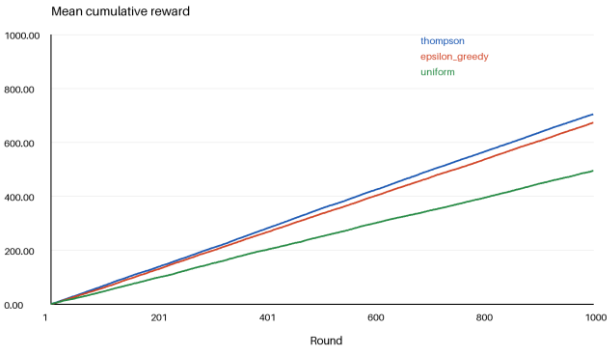


图 4 三种探索策略的平均累计奖励。

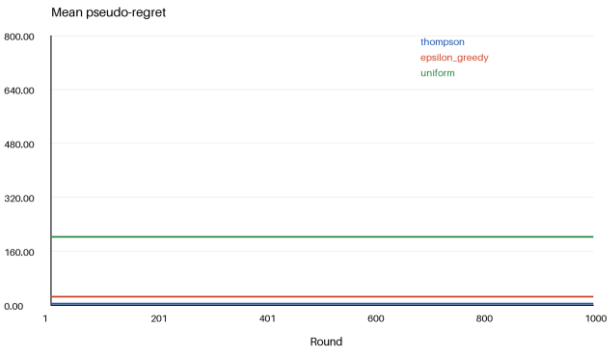


图 5 三种探索策略的平均伪遗憾。

E. 做梦的后续收益与错误泛化

表 4 做梦候选验证与后续复用的配对结果

条件	正确/320	错误	后续读源	验证访问
关闭做梦	320	0	320	0
未经验证复用	200	120	0	0
验证后复用	320	0	160	40

验证条件接受 10 个线性候选、拒绝 10 个饱和族候选。线性族在后续 160 题中复用规则；饱和族 160 题回退来源。后续读源由 320 降至 160，减少 50%；计入 40 次验证访问后为 200，相对 320 减少 37.5%。若同时计入每条件相同的 60 次训练访问，则从 380 降至 260，减少 31.58%。未经验

证条件在饱和族每轮三个超出阈值的输入上出错，共 10×4×3=120 次，表明压缩成规则可能损失边界。验证后复用的正确率与关闭条件相同，收益表现为减少取证，不能称为推理准确率提高。

被接受候选通过真实 record_practice_outcome 写回，第一次成功仍为 candidate，第二次成功后进入 warm。该路径把做梦候选、工具验证和后续实践连接起来；后续反馈的参考值由同一合成任务生成器计算，评测器仍可访问参考值，不能将其解释成真实部署中的免费外部证据。实验中的归纳模板由宿主预先指定，实际脚本模型调用数为 0；同种子重复轮次不视为独立统计样本。结果说明本协议中验证的作用，尚不能估计真实宿主自由归纳的成功率、token 成本或跨领域泛化。

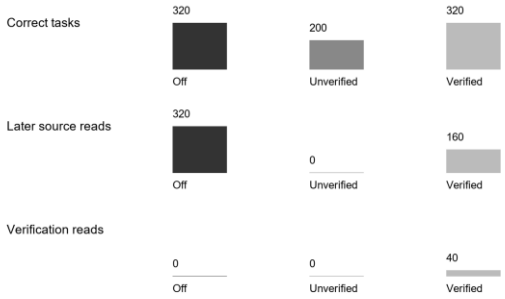


图 6 做梦三条件的正确任务数、后续读源与验证访问。

F. 端到端门槛实验：经验形成与任务收益的连接

表 5 六条条件的首次结果与门槛代价（20 种子合计，每条件 2880 题）

条件	正确	错误	未知	源读取	写回	闸拒绝
G0 精确索引	2759	0	121	2759	0	0
G1 直接写 warm	2880	0	0	496	496	0
G2 生产门槛	2880	0	0	736	736	240
G3 门槛加候选坏主张	2844	0	36	992	992	449
G4 无闸且候选可复用	2653	209	18	752	943	0
G5 门槛加双证据坏主张	2634	209	37	1601	1792	868

复现自检给出两项结果：G1 在 10 个原种子上的合计为 1440 次正确与 253 次源读取，与表 1 中 B2-full 逐行相同；任务流与已发布档案逐项相同，同机重复运行的 864 行逐字段相同。因此表 1 的 4.51 个百分点是在“源规则首次取到即写 warm”的捷径下取得的，它绕过了候选门槛与第二次独立实践；把门槛放回循环后，G2 相对 G0 的增益是 4.20 个百分点（每种子多完成 6.05 题，种子簇 bootstrap 95%区间[3.65,4.79]个百分点，20 个种子全部为正）。两个数字量级接近但含义不同：前者是“已经有经验之后”的上界，后者包含门槛成本。

门槛的成本是一条恒等式：G2 比 G1 每种种子多读 12.0 次来源（区间[12,12]），恰好等于其每种种子 12.0 次闸拒绝，而两者正确率完全相同（20 个种子并列）。也就是说，门槛没有拿走正确率，只把每次“命中了但状态不允许”换成一次源读取。相对无经验的索引基线，G2 少读 2023 次来源并多完成 121 题，即读取减少 73.33%。

状态闸的作用被收窄到一个可证的点。同一条坏主张（256 条注入）在 G3 上首次答错 0 次，在 G4 上产生 209 次错误且 256 条全部至少骗到一次（每条目中位 1 次），在 G5 上错误次数与 G4 完全相同。也就是说，补上第二次独立实践没有减少坏主张骗人的次数：闸拦住的是“只带一条证据印象、尚未完成第二次独立实践的主张”，而不是“有证据的坏主张”。闸的代价则以覆盖的形式出现：G3 的 36 次未知全部落在闸拒绝且来源不可用的第 6、7 轮，它把“可能答错”换成了“答不出”。

反馈通道存在单向性。探针显示 warm 记忆在一次失败后进入 disputed，此后即使再成功两次也不回升为 warm，只有显式修订才能重开；主实验中 191 次“命中、答错、当场纠正”全部落在这段单向区间（G4 落 candidate，G5 落 disputed）。这不是实现错误，而是升级条件含 failures 等于 0 的直接后果，本文把它记为待解限制。

G. 结果解释

主实验支持“已知规则可以在条件匹配时复用”的工程结论，没有证明经验主张如何由实践自动生成。B1 只在首轮缓存规则，后续遇到新版本仍可读取来源，但不再缓存新规则；它的局限是重复取证，而非必然输出旧版本错误。候选形成、验证门槛和失败争议原本只在接口控制里覆盖；§VI.F 的端到端门槛实验把它们与任务收益连成同一份账本，结论是门槛的作用真实存在但范围很窄：它拦住未完成第二次独立实践的坏主张，不提高正确率，也不拦住带两条证据的坏主张。Bandit 控制的收益来自探索样本分配，与主实验中的实践证据生命周期是不同层次。两组数据不能合并为一个模型效果数字。

H. 真实宿主多轮实验

前三项实验的宿主或是确定性求解器，或是单轮非盲试跑。为补齐“真实模型 + 真实工具 + 多轮 + 客观判分”，另立预注册协议（写于任何宿主作答之前）：本会话的模型经标准输入输出上的 JSON-RPC 直接调用召回与读源两个工具，只有“取经验、读源、提交答案”三个动作；经验写回由运行框架在精确匹配判分之后执行，宿主没有写经验的通道。12 个实体按 3 族 × 4 划分，每族前 2 个三版本不变、后 2 个逐轮升级，共 36 次作答，属 pilot，不报 p 值。

五条预注册判据全部成立。稳定实体首轮与末轮各 6/6，正确率不下降；漂移实体在后两轮“沿用旧版本规则”导致的错误为 0，且每轮读源 6 次恰等于漂移实体数；稳定实体在后两轮的源读取降为 0 而正确数不减；末轮时 6 条稳定记忆为 warm（各 3 次独立实践、3 个独立实践键），18 条漂移记忆按版本裂成三条、各停在 candidate；36 次作答全部有召回归档，未读源的经验作答其所用记忆均能在该轮召回结果中找到。源读取合计 24 次，相对“每轮每实体都读源”的 36 次省下 33.3%，且省下的全部落在稳定实体的后两轮。

三条限制必须同时给出。其一，正确率饱和（36/36），判据的“不下降”是平局成立而非提升，可读信息全在行为侧：版本准入是宿主自己比对召回内容中的 version 字段后主动执行的，若偷懒复用旧规则，后两轮各会多错 6 题。其二，宿主非盲且每题四则运算由本地脚本完成，本实验证明“通道、版本准入、复用”这条链路在真实宿主驱动下成立，不证明模型推理能力，也不能跨模型外推。其三，运行前修掉两处会让题目本身不可答的构造缺陷：规则原先按轮生成，导致同一实体同一版本在归档里出现三条不同 RULE 行；归档亦未写明换算公式。修掉之后宁可判据不成立，也不以不可答的装置宣称结论。该实验另暴露一处接口短板：召回只做子串模糊匹配，没有按实体或标签的结构化过滤，limit=10 时 12 条记忆只回 10 条且可能恰好漏掉提问实体本身，需宿主放大上限后自行过滤。此项记为待改接口，不记为机制结论（该接口已于 2026-11-02 修复，修后复算见 §VI.I 末段）。

I. 真实宿主压力档实验（近邻版本与坏经验）

上一小节的 36 次作答全部正确，正确率饱和使它没有分辨力：写回的记忆内容本身就是判分后的真值，复用正确只说明"抄对了"；漂移实体的版本号还写进了字段名与端点名，肉眼即可辨认。为把装置变成有分辨力的，另立预注册协议（同样写于任何宿主作答之前），只加两个压力轴，其余口径与上一小节完全相同。其一，近邻版本诱饵：同一实体相邻版本的字段名与端点名完全不随版本变化，数值只差 3%（offset 逐版本加 3，divisor 与 factor 取 100/103/106/109），版本只能靠 RULE 行里的 version 字段区分。其二，上游坏经验：3 个实体的记忆在宿主第一次召回之前就已是 warm，内容是错值（偏移 7 或 37），并带两条独立成功观察，模拟上一批会话把提取错误写进经验库。规模扩到 18 个实体、4 轮、72 次作答。

五条预注册判据全部成立。洁净稳定组 6 个实体四轮 24 题全对，后三轮源读取为 0；含诱饵的 9 个漂移实体在后三轮每轮读源 9 次、旧版本错误 0，这说明上一档"版本号写在字段名里"并不是它不出旧版本错误的原因，真正起作用的是 version 字段被显式比对。坏经验组 3 个实体在首轮全部被盲信并判错（3/3），判错后状态立刻从 warm 降为 disputed，宿主在下一轮即改读源、三题全对，恢复轮次为 1。源读取合计 51 次，相对满读 72 次省 29.2%，但这 21 次要拆开记：18 次来自洁净复用，3 次是盲信坏经验换来的（代价是 3 道错答），坏经验另在随后三轮多付 9 次读源。只报 29.2% 会把一次失败读成收益。

最值钱的一条读数在记忆终态上：三条被写坏的记忆最终各带 5 次成功、1 次失败观察，状态停在 disputed，而内容一个字符都没有变。记忆内容只在创建时写入，成功实践只调状态与置信度，因此这三个实体永久失去复用价值。据此可以说清机制的边界：判分、失败观察、降级、改策略这条链是通的（能止损），但没有内容级冲突检测与纠错通道（不能自愈）；要改内容只能另立一条新记忆或走做梦与人工修订通道，本档协议没有开放这两个动作。必须同时给出的限制是：宿主非盲且知道本档有坏经验组，坏经验是人工注入、偏移量已知，72 次作答中只有 3 次错误，仍属 pilot；算术由本地脚本执行，本档证明的是通道与策略链，不是模型推理能力。

这两处短板在同一天被修掉，并用同一批产物做了修后复算。召回改为「稀有长词闸门 + IDF 排序」：每个词按它在成员自有记忆里出现的次数算权重，只有长度不少于四个字符、且在库里稀有的词（文档频率不超过记忆总数的四分之一）才当硬闸门，其余词只参与排序，返回条目附带 match_score、matched_tokens 与 gated_by 三个可审计字段。以本档 45 条真实记忆、4 轮 × 18 个实体共 72 次查询复算，单次返回条数中位由 45 条（整库）降到 2.5 条（稳定实体 1 条、漂移实体 4 条，版本判断仍由宿主做），与提问实体无关的条目由 3060 条降到 0，精确率由 5.56% 升到 100%，72 次查询无一返回空。闸门只在查询确实含库内稀有长词时生效；问库里根本没有的实体则退回普通或链，宁可多给一条同族经验也不装作没经验。内容纠错补上 butian_memory_revise：改写必须带这条记忆未用过的新证据，旧版本标 superseded，事件表记 content_revision 与新老内容哈希，成功与失败计数从修订时刻重新起算，于是「纠正—重新验证—回到 warm」这条路走得通；回归测试把「一次失败打成 disputed、随后三次成功仍不解锁、显式修订后两次独立成功回到 warm」固化为用例。必须同时说明两点：这两条读数是接口层修复的复算，本档实验本身仍是修复前跑的，只测到旧行为；显式修订通道也不等于自动冲突检测，系统仍然不知道记忆内容与来源是否一致。

J. 真实宿主的误伤、内容修订与规模实验

前两档把“坏经验”与“版本准入”测清楚了，但有三件事在修复前做不出来：内容正确却被误判会怎样、写坏的记忆能不能修、库里几百条记忆时还准不准。第三档协议（同样在任何宿主作答之前冻结）用 3 组 × 3 族 = 9 个实体 × 4 轮 = 36 次作答回答它们，并在库内另预置 300 条同族无关记忆。36 次作答的结果是 33 对、3 错、0 未知，3 个错全部是设计出来的那一句：修订组首轮用坏经验作答。

误伤组的三条记忆内容是对的，只是被一条失败观察打成 disputed。宿主四轮十二题全部读源、全部答对：误伤不会让答案变错，但复用收益整块归零；更硬的读数在状态上——每轮判分后都给它记一次成功观察，累计成功由 3 涨到 6，状态一次都没离开 disputed，因为转 warm 要求“成功不少于两次且失败为零”，那次失败始终在计数窗口内。因此误伤不是正确性问题，是成本问题加一把单向锁。

修订组的三条记忆内容是错值，带着两条独立成功观察停在 warm。首轮宿主直接复用，三题全错；判错后运行框架用新测量证据执行内容修订，状态由 disputed 变为 candidate，计数窗口从修订时刻重算；随后两轮候选期内宿主继续读源、全对并各记一次成功；第二次成功的当轮转回 warm，第四轮不再读源、三题全对。修复通道的代价因此可以被精确量出：一次修订、两次独立成功、中间两轮仍要读源，而不是“改完立刻可信”。

规模组除九条任务记忆外另有三百条同族无关记忆（每族一百条，均为 warm 且内容完整）。十二次召回每次只返回一条、与提问实体无关的条目为零，宿主四轮全对且四轮都没读源；同一口径在上一档是“每次四十五条、无关三千零六十条”。一处边界必须写下：查询实体在库里没有记忆时不走稀有长词闸门，退回或链兜底——试点里三个素未谋面的实体各端回一百条同族记忆，这是“宁可多给一条也不装作没经验”的有意行为，正式数据因此把规模组改成预置一条 warm 记忆。

四条限制同时给出：宿主非盲且知道三组构造；单宿主单次运行、三组各三个实体，只作行为描述，不报 p 值；误伤、坏值与三百条无关记忆都是运行框架注入的构造，不是真实事故分布；恢复节律由“成功不少于两次且无失败即转 warm”这条设计决定，不是普适常数。另有一条未测：证据本身错误时修订会把错值写得更硬，本档没有覆盖。

K. 连续漂移与版本元数据的消融对照

前三档真实宿主实验的正确率几乎饱和，§VII.D 据此写明它们只提供行为读数、不提供精度差异。一个全部答对的实验无法排除“把机制拆掉照样全对”。第四档协议（同样写于宿主看到任何一轮召回之前）第一次在真实宿主线上做出正确率差异：2 组 × 2 臂 × 3 族 = 12 个实体 × 4 轮 = 48 次作答，两臂共用同一个库、同一批以两条独立成功预置为 warm 的记忆、同一条宿主消费规则，唯一差别是召回呈现层。gated 臂原样返回 memory_id、status 与含 version 字段的内容；naive 臂只返回内容并抹掉其中的 version 字段，模拟“只存文本、检索到就塞进上下文”的记忆模块。drift 组的源文档每轮加一版而记忆全程停在 v1，stable 组四轮都是 v1 作反假绿对照。本档不做任何写回与实践观察登记，把漂移这一个因素单独隔出来。

结果给出 75 个百分点的差：漂移条件下 gated 臂 12 题全对，naive 臂只对 3 题（25%），错误 9 题；无漂移条件下两臂各 12 题全对，完全相同。差异自第一次漂移那一轮出现，并在此后每一轮稳定复现。四条预注册判据全部成立。失效形态被单独钉住：9 个错答逐个精确等于用 v1 规则算出的答案（stale_match 9/9），装置另核对每个漂移实体在 R2–R4 的旧版本答案都不等于当轮真值，因此这不是随机答错，而是沿用旧版本。naive 臂四轮一次源都没读过——呈现层里没有版本也没有状态，消费者没有任何可据以怀疑的信号；gated 臂在每次版本不一致时退回源文档，代价是 9 次读源。记忆终态核对显示漂移没有把旧记忆洗白：三个 gated 漂移实体的记忆四轮取值始终是版本 1、状态 warm。

这条读数把版本与状态元数据从“设计上应该有”推进到“拆掉就会掉正确率”。但结论的范围必须写死：naive 臂的“无元数据即信任内容”是建模假设，不是对某个真实开源记忆模块的实测，因此只能读成“在信任召回内容的消费者下，版本元数据承重”，不能读成“任何朴素记忆系统都会错”。stable 对照排除了“naive 臂本身就坏”这一替代解释，却不排除消费者侧还有别的可行策略。本档全程不写回，故只回答“会不会沿用旧版本”，不回答“能不能学会新版本”；算术仍由本地脚本执行，单种子单次运行、宿主非盲，12 条独立规则不能当作 48 个独立样本，不报 p 值。跨模型一致性由下一小节的外部模型复现回答。

L. 把同一消融交给外部模型：跨模型复现

上一小节的宿主仍是本会话中的模型，它同时是实验设计者，因此排除不了“这个差是这一个宿主的行为习惯”。第五档协议（同样冻结于任何外部模型看到任何一条召回之前）把宿主换成外部 API 模型，任务、规则、记忆、召回呈现层与消费规则全部沿用上一档，只换宿主，两个模型各 48 题、合计 96 次作答。其中 DeepSeek-V4-Pro 与会话宿主不同族，是真正的跨模型臂；DeepSeek-V4.1-Flash 与会话宿主同族，只作独立进程复现、不承担跨模型结论。外部模型不知道分组构造、不知道假设、不参与设计，上一档的“宿主非盲”这一条在本档被消除。

两个外部模型逐格复现了上一档：漂移组 gated 臂 12/12、naive 臂 3/12，稳定组两臂各 12/12，读源次数同为 9 与 0，四格读数与会话宿主完全相同。四条预注册判据在两个模型上全部成立，判词 v5_cross_model_replicated。漂移后三轮的两臂差都是 100 个百分点，naive 臂的 9 个错答仍然逐个精确等于旧版本规则的答案。副口径单独判分模型自己算出的答案，两个模型都是 39/48 且与主口径逐格相同——9 个错答全部来自选错规则版本，没有一题是选对规则却算错数，算术能力不是本次差异的来源。每个模型 57 次 API 调用（48 次决策 + 9 次读源后作答），JSON 解析失败 0、协议错误 0，完整请求与回复逐次落盘可核。

协议还提前写死了一处装置缺陷并预注册了干净子群：naive 呈现层只抹掉显式 version 与 memory_id、status，没有抹掉内容里偶然携带的版本线索（tool 族的 /api/v1/ 与 limit_v1、schema 族的 f1 与 h1）。calibration 族的内容只有 scale 与 offset 两个纯数值，是唯一不含任何版本字符的子群。实测该子群的两臂差在两个模型上同样是 100 个百分点，逐族分解也完全一致。这条读数把结论从“可能靠端点串里的 v1 认出来”收窄为：即使内容里一个版本字符都没有，抹掉元数据照样让模型沿用旧规则。范围仍须写死：两个外部模型同属一家厂商，不能外推为任意模型；消费规则仍是写进提示词的建模假设；设计者非盲仍在；每宿主单次运行、temperature=0、无重复采样，不报 p 值。

M. 用与不用：经验库的双轴有效性

前五小节都在问“补天里的某个元件是否承重”，两只臂都在用经验库，因此只能证明元件必要，证明不了整体上比不用更好：正确率一旦饱和，拆零件掉的分只能说明零件有用。第六档协议（同样冻结于任何外部模型看到任何一条召回之前）换掉问法，直接对比两条真实工作流：lib 臂先查经验库、只在记忆不可信时读源；noref 臂完全没有记忆层，每题只能读源。语料是一份 327 行、38063 字节的合成设备手册，覆盖标定、协议、工具三族、每族 40 个实体共 120 个实体，每实体 1 至 4 个版本行；任务取每族 8 题共 24 题，其余 96 个实体作诱饵，两臂都不知道哪些会被问。经验库预置全部 120 条、每条带两份独立证据并处于 warm，因此召回既可能命中任务实体，也可能命中诱饵或旧版本。漂移分三档：第一轮全为 v1，第二轮 8 个实体

改版，第三轮这 8 个仍是新值、另加 4 个首次改版，陈旧比依次为 0/24、8/24、12/24；改版时强制重抽参数，保证“沿用旧版本”与“正确答案”不同值。两臂共用同一语料、同一任务顺序、同一提示词骨架，temperature 为 0，每题独立上下文、不跨题缓存。

第一轴看成本。不限额度、三轮共 72 题时，lib 臂与 noref 臂的正确率分别为 98.6%与 100%（DeepSeek-V4-Pro）以及 100%与 100%（DeepSeek-V4.1-Flash）；lib 臂读源 20 次，noref 臂 72 次，节省 72.2%，提示词 token 由 1054132 降到 335918（降到 31.9%），Flash 宿主由 1021611 降到 331183（降到 32.4%）。两宿主读源次数完全相同，因为该不该读由冻结的漂移日程决定，而不是由模型偏好决定。Pro 宿主唯一的失手是第三轮一题读到了正确来源却输出无法解析的答案，被判 unknown，属格式抖动而非记忆层缺陷。

第二轴看预算。给每臂每轮只有 6 次或 12 次读源额度、题量仍是 24 题时，lib 臂完成 18/24 与 24/24，noref 臂只完成 6/24 与 12/24，差 50 个百分点，两个宿主逐格相同。noref 臂的读数与额度一一对应：每读一次手册只能答一道题，额度用完即停工；lib 臂则把额度只花在“记忆里那条已不可信”的题上，6 次额度恰好覆盖 6 个陈旧实体、12 次覆盖全部 12 个。把“该不该读源”当作二分类闸门逐题核对：三轮陈旧题数为 0、8、12，lib 臂实际读源 0、8、12 次且逐题与陈旧集合完全相同，零误花、零漏读，两个宿主都是 24/24。

最值得单独写下来的是失败形态。第二轴额度耗尽、记忆又是旧版本的那几道题，lib 臂全部返回 unknown，没有一次凭旧值瞎猜；noref 臂额度用完后同样返回 unknown 而不编数字。因此本档 noref 臂输的不是“答错”而是“停工”，25%与 50%是能干完的比例，其余是明确表示做不了。这与“宁可报错不给错答案”的要求一致，也让增强的含义更诚实：装配经验库的收益主要兑现为同等预算下完成更多工作（3 倍与 2 倍），而不是把错答改对。

范围须写死。手册是合成的，条目单一、取值唯一、无歧义，真实文档的歧义、冲突与缺页没有覆盖；陈旧标记由补天自身计算，本档没有注入“标记本身出错”的情形，标记一旦不可靠，24/24 的闸门会退化为误读或漏读；每题独立上下文使 noref 臂的读源成本被高估，该偏差方向对 noref 不利；每档只有 24 题，50 个百分点的差远超噪声，但绝对数值依赖语料构造，不可外推为任意任务上的百分比；本档只测消费不测写回，不测多轮指代与记忆污染，召回每题只命中 1 条，是 limit 为 20 的理想情形；两个宿主同属一家厂商，只做到同族与跨族各一个点。

N. 自己攒经验：形成侧的有效性

前六档的经验库都是实验者预置的：120 条、每条两份独立证据、状态 warm，模型只负责消费。这只能证明“有库比没库强”，证明不了库由宿主自己长出来之后是否还有用——真实部署里没有人替它预置。第七档协议（同样冻结在任何宿主看到任何一条召回之前）把库清空：form 臂（召回 + 读源 + 建档 + 改档）从空库起步，noref 臂只有读源；语料、任务、漂移日程与第六档完全相同，单宿主 DeepSeek-V4.1-Flash，temperature 为 0，每题独立上下文。

两臂各 72 题。form 臂 72/72、noref 臂 72/72，正确率不降。读源 36 次对 72 次，省 50%；只看库建起来之后的第二、三轮，12 次对 48 次，省 75%。提示词 token 由 1021179 降到 604974，为对手的 59.2%。三轮的决策分得很干净：第一轮 24 题全部读源建档；第二轮手册改版的 8 个实体读源、其余 16 题直接复用；第三轮手册陈旧实体增至 12 个，但其中 8 个在第二轮已按新值改档，相对库而言仍属未知的只有 4 个，于是读源 4 次、复用 20 题。省下的正是“库里有、版本对、状态可用”这一类题。

形成不是免费的。写回动作 practice 24 次（首次建档，独立键由源行决定）、revise 12 次（与手册实际改版逐条对应）、unchanged 与 skipped 与 error 均为 0；状态机走过 candidate 被复用到 superseded 换新链的完整路径。第一轮是纯投入：48 次调用对 24 次，这一轮比对手贵，收益从第二轮才开始兑现，因此 50% 是三轮平均，不能读成“一上手就省”。污染与陈旧复用各 0 次：装置不纠正写回内容，写回的每一行都逐字来自源行；12 个版本不一致的题全部转去读源而不是硬用旧值，36 个版本一致的题全部直接复用，闸门 48/48 逐题正确。

范围须写死。本档只有单宿主、单种子、24 个实体、3 轮，不能外推为任意模型或任意文档；协议 §7.7 明确允许自抄 candidate 复用（同一份源读两遍不构成独立证据，若强按“两次独立成功才转 warm”的默认规则，形成成本会吃掉全部收益），这是建模设定而非补天默认行为，代价由污染判据兜；每个实体与版本组合的源行由装置单独登记为证据件、写回使用真实证据标识，这是装置保证，不构成模型能力；语料仍是合成的、无歧义的合成手册，仍未注入“标记本身出错”的情形。

VII. 讨论：从人的实践逻辑到宿主智能体机制

A. 这套模型模拟了什么

模型模拟的是人的实践结构，而不是人的全部心理过程。它保留目标、行动、结果、条件、证据、反例、修订和再实践这些可以在工程中观察的部分。毛泽东《实践论》关于认识来源于实践并在实践中检验的思想可作为研究设计的哲学指导[7]，但哲学思想不是程序正确性的证据；程序正确性仍由状态守卫、事务和实验结果支持。已有长期记忆研究也提示恢复时必须携带任务情境，但软件字段与人的记忆不能直接等同[1-2]。

B. 贝叶斯不是经验本身

时间上下文模型讨论了记忆提取与变化情境的关系[8]，本文仅借此说明恢复应携带环境，而不以认知模型证明 SQLite 结构等同于人类记忆。Thompson Sampling 的教程讨论了根据后验采样进行探索的框架[9]；本文固定两臂控制仅覆盖其中最简单的平稳 Bernoulli 情形，不能把该结果推广到真实宿主行动的非平稳、部分可观测和延迟反馈环境。

Beta 后验回答“在当前工作模型下，证据如何改变可信度摘要”，不回答“人是否已经真正理解”，也不替代反例、版本和来源。贝叶斯更新是经验生命周期中的一个数值部件。Thompson Sampling 回答“多个待验证行动如何分配探索机会”，不是经验形成的全部，也不是当前项目已经实现的完整强化学习。因此，探索分配器的性能与经验生命周期的有效性需要分别评估。

C. 与宿主 AI 和模型训练的关系

运行期经验库改变的是可召回上下文、证据和策略选择，不是模型参数。宿主 AI 可以读取候选、执行行动、提交反馈，但其自由文本回答必须单独记录首次答案、可见

上下文、工具结果和反馈。当前 15 条同 Agent 非盲探索不能作为独立模型实验；主实验 10080 次也没有语言模型调用。经验可成为后训练候选资料，但要改变参数必须有训练前后固定评测、遗忘评估和回滚证据，本文没有把运行期写回称为自进化。

D. 有效性威胁

任务族是契约化合成任务，确定性求解器降低了模型噪声但也限制了外部有效性；10 个种子是统计簇，不能把 1440 题当 1440 个独立样本；漂移和断源日程是协议内设定，不代表真实世界分布；本地时延不等于网络调用或模型 token 成本；同 Agent 试跑存在先前答案暴露。接口回归检查用于验证实现行为，不纳入任务收益统计。端到端门槛实验把形成与收益放在同一账本上，但其坏主张是实验者施加的固定冲击，不是生产中出现频率的估计；20 个种子中 G1 与 G2 的正确率都达到 100%，该装置对正确率不敏感，可分辨的只有源中断两轮的覆盖、读取代价与注入后的首答错误。真实宿主多轮实验的非盲问题更直接：宿主同时是实验设计者。前三档共 144 次作答中只有 6 次错误，且那 6 次都是设计出来的坏经验首答，因此这三档只提供行为读数（版本准入被主动执行、复用使稳定实体读取归零），不提供机制层面的精度差异；其四则运算由本地脚本执行，也不能读作模型推理能力的证据。第四档的消融对照补上了这一缺口：抹掉召回的版本与状态元数据后，连续漂移下的正确率由 12/12 跌到 3/12，无漂移对照组两臂各 12 题全对。该差异仍受两点限制：朴素臂“无元数据即信任内容”是建模假设而非对真实记忆模块的实测，且它仍是单宿主单次运行的非盲实验。第五档把宿主换成两个外部 API 模型（其中一个与会话宿主不同族）后，四格读数逐格复现、干净子群同样成立，宿主非盲这一条被消除；但两个外部模型同属一家厂商，消费规则仍是写进提示词的建模假设，设计者非盲仍在，因此不外推为任意模型。第六档不看元件而看工作流本身：装配经验库的一臂在不限额下以 20 次读源完成 72 题（对手 72 次）、提示词 token 为对手的 31.9%而正确率不降；两臂各给 6 次与 12 次读源额度时，完成率分别为 75%对 25%与 100%对 50%，两个跨族宿主逐格相同，闸门 24/24 零误花零漏读，额度耗尽时返回 unknown 而不猜测。该档的未测前提同样明确：陈旧标记由本系统自算，标记出错的情形没有覆盖。第七档把库清空、由宿主自己建档：两臂各 72 题正确率不降，读源由 72 次降到 36 次（库建成后的两轮由 48 次降到 12 次），提示词 token 为对手的 59.2%，建档 24 次与改档 12 次和手册实际改版逐条对应，污染与陈旧复用各 0 次；其未测前提是单宿主单种子，且“自抄 candidate 可复用”是协议内写明的建模设定。压力档实验把同一问题推进一步：注入带两条独立证据的坏主张后，宿主在首轮盲信并判错，随后依据状态降级改读源，而被写坏的记忆内容始终没有被修正——状态级降级能止损，不能自愈。

VIII. 结论与后续工作

本文把人的实践经验形成过程明确建模为：目标驱动行动，行动产生观察，观察与预期比较形成反馈，反馈经来源和条件验证后成为候选经验，候选经验通过重复实践和反例检验进入稳定或争议状态，并在版本变化和重启后继续接受验证。补天的数据库、证据、版本链、上下文恢复和反馈接口分别实现了这条过程的可观测环节。

实验支持五条限定结论：第一，在契约化任务中，带版本准入的规则复用策略比精确索引基线少 81.60%源读取；取消版本准入的对照出现 133 次旧版本错误；第二，持久化使重启后的经验可以继续参与召回；第三，Beta 更新和 Thompson 控制在合成数值环境中符合既定公式；第四，把候选门槛放回循环后，端到端门槛实验在 10 个原种子上逐行复现了 B2-full 的 1440 次正确与 253 次读取，并在 20 个种子上给出门槛的净效应：相对精确索引多完成 121 题、少读 2023 次来源（73.33%），代价是每种种子多读 12 次；状态闸拦住了只带一条证据的坏主张（0 次错误对 209 次错误），却拦不住带两条独立实践的坏主张。新增做梦回放中，验证后复用与关闭条件均完成 320 题，计入验证后的取证访问减少 37.5%；未经验证复用产生 120 次错误泛化。该结果支持在这两类任务中先求证后复用，不能推广为自由文本自主归纳收益。第五，把同一流程接到真实宿主后，五条预注册判据在 36 次作答上全部成立：稳定实体后两轮源读取归零、漂移实体零次沿用旧版本错误、稳定记忆升为 warm 而漂移记忆按版本停在 candidate；但 36 次全部答对意味着该实验只提供行为读数，不提供精度差异。把装置加上压力轴后（近邻版本诱饵与上游坏经验，18 个实体、4 轮、72 次作答），洁净组仍 24 题全对且后三轮读取为零，诱饵组零旧版本错误，坏经验组首轮 3 题全错、次轮即全部恢复，但被写坏的记忆内容始终未被修正：该条记忆在随后三轮又拿到三次成功（终态 5 成功 1 失败）仍停在 disputed，说明状态门槛能止损、不能自愈。第四档把版本与状态元数据单独消融：同一库、同一批记忆、同一条宿主消费规则，只把召回呈现层里的版本与状态抹掉，连续漂移下正确率就由 12/12 跌到 3/12，9 个错答逐个精确等于旧版本规则的答案，而无漂移对照组两臂各 12 题全对；这是本文中真实宿主线上的第一处精度差异，把元数据从设计要求推进到可被拆检验的承重件，但其朴素消费者行为是建模假设，不构成对真实记忆模块的实测。第五档把同一消融交给两个外部 API 模型重跑 96 次作答，漂移组 12/12 对 3/12、稳定组两臂各 12/12 的四格读数逐格复现，失效形态仍是精确沿用旧版本，且在不含任何版本字符的 calibration 子群上差异同样为 100 个百分点；模型自算答案的副口径与主口径逐格相同，排除了算术能力这一混淆。宿主非盲由此被消除，但两个外部模型同属一家厂商，不外推为任意模型。第六档把比较对象从“拆零件”换成“用与不用”：装配经验库的一臂在不限额下以 20 次读源完成 72 题而对手需要 72 次，提示词 token 为对手的 31.9%，正确率不降；两臂各给 6 次与 12 次读源额度时，完成率分别为 75%对 25%与 100%对 50%，差 50 个百分点，两个跨族宿主逐格相同，闸门 24/24 零误花零漏读，额度耗尽时返回 unknown 而不猜测。这把结论由“元件承重”推进到“同等代价下完成更多工作”，但仍不外推为任意语料与任意模型：手册是合成且无歧义的，陈旧标记由本系统自算，标记出错的情形未测。第七档进一步把库清空，由宿主自己建档与改档：两臂各 72 题正确率相同，读源由 72 次降到 36 次、仅看库建成后的两轮由 48 次降到 12 次，token 为对手的 59.2%，建档与改档次数和手册改版逐条对应，污染与陈旧复用各 0 次；该档的代价是第一轮纯投入（48 次调用对 24 次），收益从第二轮起兑现。代价是更多逻辑操作和本地时延。本文不声称模型参数自进化、完整 Bayesian RL 或普遍语言模型收益。后续应把同一实践流程接入真实宿主 AI，按首次答案冻结、客观工具结果、反馈、复用和反例逐轮

记录，以检验人的实践模型是否真正改善宿主 Agent 在同类问题上的任务结果。

参考文献

[1] Wanjun Zhong, Lianghong Guo, Qiqi Gao, et al. MemoryBank: Enhancing Large Language Models with Long-Term Memory[EB/OL]. arXiv, 2023. http://arxiv.org/pdf/2305.10250v3.

[2] Di Wu, Hongwei Wang, Wenhao Yu, et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory[EB/OL]. arXiv, 2024. http://arxiv.org/pdf/2410.10813v2.

[3] M. T. Howard, "Experiential learning – a systematic review and revision of Kolb's model," Interactive Learning Environments, 2020, doi: 10.1080/10494820.2019.1570279.

[4] Noah Shinn, Federico Cassano, Edward Berman, et al. Reflexion: Language Agents with Verbal Reinforcement Learning[EB/OL]. arXiv, 2023. http://arxiv.org/pdf/2303.11366v4.

[5] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models[EB/OL]. arXiv, 2023. http://arxiv.org/pdf/2305.16291v2.

[6] Patrick Lewis, Ethan Perez, Aleksandara Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[J], 2020.

[7] 毛泽东, '实践论,' 毛泽东选集, 第 1 卷. 北京: 人民出版社, 1991, pp. 282–297. 原作 1937 年 7 月; 高校电子转载核验.

[8] H. M. W., K. M. J, "A Distributed Representation of Temporal Context," Journal of Mathematical Psychology, 2002, doi: 10.1006/jmps.2001.1388.

[9] R. D. J., V. R. Benjamin, K. Abbas, O. Ian, W. Zheng, "A Tutorial on Thompson Sampling," 2018, doi: 10.1561/9781680834710.

附录 A：实现映射与复现实验

核心实现与正文映射如下：经验存储负责保存当前版本、父版本和来源事件；实践记录负责处理成功、失败和争议转移；上下文组装与解析负责候选经验召回、实体版本和完整性准入；备份与恢复负责服务状态和证据路径；数学控制部分实现 Beta、信息增益、效用和 Thompson 采样。本文直接给出关键模块、算法、约束和实验口径，不以未公开的静态台账代替机制解释。

本文报告的实验由确定性机制回放、生命周期对照、数学控制和做梦效果对照组成；主实验与控制实验均未调用语言模型。做梦效果对照使用宿主预先提供的候选模板，并通过真实的验证与记录接口产生证据，因此不是自主发现或独立模型实验。当前文档报告的是机制证据，不能替换尚未完成的真实宿主 AI 确认性实验。

附录 B：实验图表

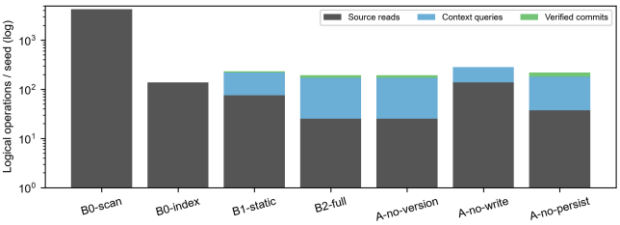


图 7 实践过程中的取证、召回和写回操作。

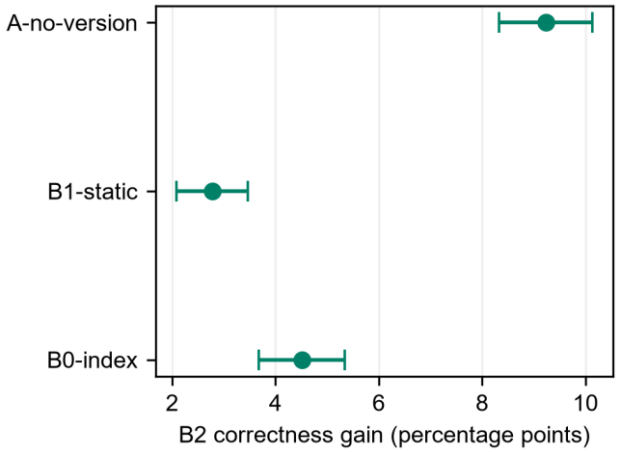


图 8 种子簇配对效应。